

Diplomatura en programación web full stack con React JS

Módulo 3:

Javascript y maquetado avanzado

Unidad 4:

Javascript (parte 3)



Presentación

En esta unidad vemos los distintos métodos de manipulación de arrays y objetos en Javascript.



Objetivos

Que los participantes logren...

- Explorar distintos métodos de manipulación de arrays.
- Comprender las estructuras de objetos de JS.
- Dominar la desestructuración de objetos.



Bloques temáticos

1. Métodos de array: map, filter y find.
2. JSON.
3. Destructuring y operador spread.

1. Métodos de array: map, filter y find

Array.map()

El método **map()** crea un nuevo array con los resultados de la llamada a la función indicada aplicados a cada uno de sus elementos.



```
1 const numeros = [1, 3, 8, 20];
2 const dobles = numeros.map(function(num) {
3     return num * 2;
4 });
5
6 console.log(dobles); // [2, 6, 16, 40];
7
8 // versión simplificada usando arrow functions
9 const dobles = numeros.map(num => num * 2)
```

Array.filter()

El método **filter()** crea un nuevo array con todos los elementos que cumplan la condición implementada por la función dada.



```
1 const numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];  
2 const pares = numeros.filter(num => num % 2 === 0);  
3 console.log(pares); // [2, 4, 6, 8, 10]
```

Array.find()

El método **find()** devuelve el valor del primer elemento del array que cumple la función de prueba proporcionada, de lo contrario devuelve undefined.



```
1 const numeros = [3, 5, 10, 15, 28, 76, 149];  
2 const encontrado = numeros.find(num => num > 12);  
3 console.log(encontrado); // 15
```

Array.sort()

El método **sort()** ordena los elementos de un arreglo (array) localmente y devuelve el arreglo ordenado. La ordenación no es necesariamente estable. El modo de ordenación por defecto responde a la posición del valor del string de acuerdo a su valor Unicode.

```
const frutas = ["guindas", "manzanas", "bananas"];
frutas.sort(); // ['bananas', 'guindas', 'manzanas']

const puntos = [1, 10, 2, 21];
puntos.sort(); // [1, 10, 2, 21]
// Tenga en cuenta que 10 viene antes que 2
// porque '10' viene antes que '2' según la posición del valor
// Unicode.

const cosas = ["word", "Word", "1 Word", "2 Words"];
cosas.sort(); // ['1 Word', '2 Words', 'Word', 'word']
// En Unicode, los números vienen antes que las letras mayúsculas
// y estas vienen antes que las letras minúsculas.
```

Si se provee **compareFunction**, los elementos del array son ordenados de acuerdo al valor que retorna dicha función de comparación. Siendo **a** y **b** dos elementos comparados, entonces:

- Si **compareFunction(a, b)** es menor que 0, se sitúa **a** en un índice menor que **b**. Es decir, **a** viene primero.

- Si **compareFunction(a, b)** retorna 0, se deja **a** y **b** sin cambios entre ellos, pero ordenados con respecto a todos los elementos diferentes. Nota: el estandar ECMAScript no garantiza este comportamiento, por esto no todos los navegadores (p.ej. Mozilla en versiones que datan hasta el 2003) respetan esto.
- Si **compareFunction(a, b)** es mayor que 0, se sitúa **b** en un índice menor que **a**.
- **compareFunction(a, b)** siempre debe retornar el mismo valor dado un par específico de elementos a y b como sus argumentos. Si se retornan resultados inconsistentes entonces el orden de ordenamiento es indefinido.

Entonces, la función de comparación tiene la siguiente forma:

```
function compare(a, b) {  
  if (a es menor que b según criterio de ordenamiento) {  
    return -1;  
  }  
  if (a es mayor que b según criterio de ordenamiento) {  
    return 1;  
  }  
  // a debe ser igual b  
  return 0;  
}
```

Para comparar números en lugar de strings, la función de comparación puede simplemente restar b de a. La siguiente función ordena el array de modo ascendente:



```
const compareNumbers = (a, b) => a - b;
```

El método **sort** puede ser usado convenientemente con function expressions (y closures):



```
const numbers = [4, 2, 5, 1, 3];  
  
numbers.sort(function (a, b) {  
  return a - b;  
});  
  
console.log(numbers); // [1, 2, 3, 4, 5]
```

Podemos utilizar también el método sort para ordenar arrays de objetos utilizando alguna de sus propiedades.

```
const equipos = [  
  { nombre: "Leones del Sur", puntos: 25 },  
  { nombre: "Águilas Rojas", puntos: 32 },  
  { nombre: "Lobos Grises", puntos: 18 },  
  { nombre: "Halcones Dorados", puntos: 29 },  
  { nombre: "Fénix Azules", puntos: 22 },  
  { nombre: "Tigres Negros", puntos: 35 },  
  { nombre: "Dragones Verdes", puntos: 15 },  
  { nombre: "Serpientes Plateadas", puntos: 28 },  
  { nombre: "Condores Blancos", puntos: 20 },  
  { nombre: "Jaguares Púrpura", puntos: 30 }  
]  
equipos.sort((a,b) => {  
  if(a.puntos > b.puntos) {  
    return 1;  
  }  
  
  if(a.puntos < b.puntos) {  
    return -1;  
  }  
  
  if(a.puntos === b.puntos) {  
    return 0;  
  }  
})
```

2. JSON

JSON o JavaScript Object Notation es un formato de texto utilizado para definir estructuras de objetos y listas complejas. Es ampliamente utilizado para el intercambio de datos entre aplicaciones debido a la simplicidad de su implementación y la facilidad de lectura por parte de las aplicaciones y las personas. Es un subconjunto de la notación de objetos literal de Javascript, por lo cual existen múltiples similitudes entre ambos.

Estructura básica

```
1 {  
2     "nombre": "Valeria",  
3     "edad": 28,  
4     "esEstudiante": true,  
5     "hobbies": [  
6         "fútbol",  
7         "programación",  
8         "cine",  
9     ]  
10 }
```

En el ejemplo de estructura podemos apreciar lo siguiente:

- Todo el objeto está rodeado por llaves {}.
- Los nombres de las propiedades están rodeados por comillas dobles “.
- Los tipos de valores posibles son: texto, número, booleano, array o incluso otros objetos.
- Las listas o arrays se rodean con corchetes [] y pueden contener cualquiera de los tipos de datos aceptados.

En javascript podemos definir lo que se denomina objeto literal usando una sintaxis similar a JSON. Una de las principales diferencias es que no es necesario utilizar comillas dobles en los nombres de propiedades. El ejemplo anterior podría escribirse en javascript de la siguiente manera.

```
1 const persona = {
2     nombre: "Valeria",
3     edad: 28,
4     esEstudiante: true,
5     hobbies: [
6         "fútbol",
7         "programación",
8         "cine",
9     ]
10 };
11
12 console.log(persona.nombre); // "Valeria"
13 console.log(persona.hobbies.length); // 3
```

3. Destructuring y operador spread

Destructuring

La desestructuración (**destructuring**) es una técnica utilizada para extraer y declarar varias variables a la vez. Podemos aplicar esta técnica a arrays, objetos, y otros tipos de estructuras nuevas en ES6 como maps y sets.

Arrays

```
1 const colores = ["#ff0000", "#00ff00", "#0000ff"];  
2  
3 // En vez de hacer esto  
4 const rojo = colores[0];  
5 const verde = colores[1];  
6 const azul = colores[2];  
7  
8 // Podemos hacer esto  
9 const [rojo, verde, azul] = colores;
```

Objetos

```
1 const pelota = {
2   posicion:{
3     x: 150,
4     y: 150
5   },
6   colorDeRelleno:"tomato",
7   radio: 25,
8 }
9
10 // En vez de hacer esto
11 const posicion = pelota.posicion;
12 const radio = pelota.radio;
13 const colorDeRelleno = pelota.colorDeRelleno;
14
15 // Podemos hacer esto
16 const { posicion, radio, colorDeRelleno } = pelota;
```

Operador spread

El operador spread (...) sirve para obtener todas las propiedades de un objeto o los elementos de un array. Es de suma utilidad para cuando necesitamos hacer copias de objetos o listas modificando alguno de los valores o agregando nuevos.

Arrays

```
1 const vocales = ['a', 'e', 'i'];
2 const vocalesCompletas = [...vocales, 'o', 'u'];
3 console.log(vocales); // ['a', 'e', 'i']
4 console.log(vocalesCompletas); // ['a', 'e', 'i', 'o', 'u']
```

Objetos

```
1 const usado = {
2   marca: 'Chevrolet',
3   modelo: 'Corsa',
4   anio: 2012
5 };
6
7 const nuevo = {
8   ...usado,
9   anio: 2020
10 };
11
12 console.log(nuevo.marca); // Chevrolet
13 console.log(nuevo.anio); // 2020
```


Conclusión

Aprendimos los conceptos necesarios para recorrer y manipular distintas estructuras de datos utilizadas en JavaScript. Con estos conocimientos podremos incorporar a nuestras páginas datos de distintas fuentes de forma sencilla.

En la próxima unidad veremos...

React es una biblioteca de JavaScript de código abierto, que se utiliza para construir interfaces de usuario interactivas y dinámicas para aplicaciones web y móviles.



Bibliografía utilizada y sugerida

Libros y otros manuscritos:

Eguíluz Pérez, Javier. Introducción a JavaScript.España: www.librosweb.es 2008.

Artículos de revista en formato electrónico:

MDN Web Docs. (s.f.). MDN Web Docs. Mozilla. <https://developer.mozilla.org/>